

COMPUTER SCIENCE TRIPOS Part IB 75%, Part II 50% – 2021 – Paper 7

8 Further Human–Computer Interaction (afb21)

Predictive text entry systems are familiar on touch screens and mobile phones. This question asks you to consider how the same principles might be used in a programming editor for creating Java code.

- (a) Explain using Bayes’ Theorem how such an editor might update its expectation about which identifier will appear next, as the programmer is typing a line of code. Your answer should include a short extract of Java code preceding this line, in order to illustrate the context in which the expectation is being calculated.

[5 marks]

Answer:

```
public class Dog {  
    String breed;  
    int age;  
}  
  
Dog puppy = new dog();  
puppy.
```

At this point, the editor has an expectation that the next character will be the name of one of the members of the Dog class. The next character that is typed will give further information about which member. The prior expectation, before the character is typed, might be a uniform prior in which the member is equally likely to be “breed” or “age”. If the next character typed is observed to be “b”, then the likelihood that the member is “breed” given that a “b” has been typed could be updated to nearly 1, while the likelihood of “age” could be updated to nearly 0, since the it is unlikely that the programmer intends to type breed starting with the letter a.

- (b) Describe how you could obtain an empirical measurement of the actual improvement in efficiency that results from using this predictive editor in practice.

[5 marks]

Answer: I could design a controlled experiment, in which a version of the editor using the prediction function is compared to a version of the editor without it. I would recruit a sample of programmers to use both versions, and give them a controlled task in which they have to enter the same piece of source code using both versions. I would use several samples of source code, designed to be representative of real programming tasks. I would compare both the speed that they enter each piece of code, and also the number of errors in the code that they have entered.

- (c) Consider the *four* possible combinations of a) small versus large variance, and b) small versus large effect size, in repeating this measurement. For *each* combination, explain the practical interpretation of that data for future development of the programming editor.

[5 marks]

Answer: If the effect size is small and variance is small, it may be possible to report a statistically significant result from the experiment, but without much value from introducing

the new feature. If effect size is small and variance is large, then it is likely that the result will not be statistically significant. If effect size is large and variance is small, the result will be significant, and also a clear improvement in efficiency that suggests the product should be changed. If effect size is large and variance is large, it is possible that the product should be changed, but the experiment should probably be redesigned to reduce the variability in measurements. Alternatively, a within subjects experiment could be used, perhaps employing the sign test if the measurements are not normally distributed.

bounded
rationality

- (d) If programmers have the option of whether or not to turn on this new function in the editor, describe some of the factors that might influence their decision-making process, with specific reference to the cognitive processes and sources of information that they would use. [5 marks]

Answer: Programmers are unlikely to make completely optimal decisions about which features they use, because they cannot anticipate all future activities accurately, and are also focused on getting the job done adequately, rather than optimally (satisficing). As a result, they will make decisions about which features to use with the aid of heuristics and biases. One example might be the availability heuristic - if the feature is presented to them, they might select it just because it is there. Another is the representative heuristic - if the feature appears similar to other features that have saved effort in the past, they might select it because of that similarity. Programmers might also be influenced by the bandwagon effect, where a particular editor feature is considered trendy, or has had a lot of attention on a popular forum, the programmer might choose to activate it even if there is not good evidence that it will save them personally any effort.
